

Institut Supérieur d'Informatique et Mathématiques
Monastir

Bases de données NOSQL

Introduction aux bases de données NOSQL

Dernière Mise à jour:
Novembre 2024

Asma KERKENI **asma.kerkeni@gmail.com**

Références

- P. Lacomme, S. Aridhi, R. Phan, Bases de données NoSQL et Big Data: Concevoir des bases de données pour le Big Data, Cours et travaux pratiques Ellipses ~ Technosup, 2/12/2014, 336 p., 9782340002616.
- Rudi Bruchez, L. Les bases de données NoSQL et le Big Data: Comprendre et mettre en œuvre, Ed. Eyrolles, 2015.
- Cours Bases de données NOSQL de Raja CHIKY, Institut Mines Télécom: http://perso.isep.fr/rchiky/
- Cours Big Data et NoSQL de Lilia Sfaxi, INSAT, Tunisie: http://liliasfaxi.wixsite.com/liliasfaxi
- https://db-engines.com/en/ranking
- http://b3d.bdpedia.fr/
- https://chewbii.com/

Objectifs

- Au terme de ce chapitre, vous serez capable de:
  - Citer les différents types de bases de données.
  - Décrire le contexte d'apparition de la mouvance NOSQL.
  - Définir les propriétés ACID et BASE et le théorème CAP
  - Comparer les bases de données relationnelles et les bases NOSQL.
  - Enumérer et caractériser les bases de données NOSQL.
  - Caractériser les BDs NewSQL

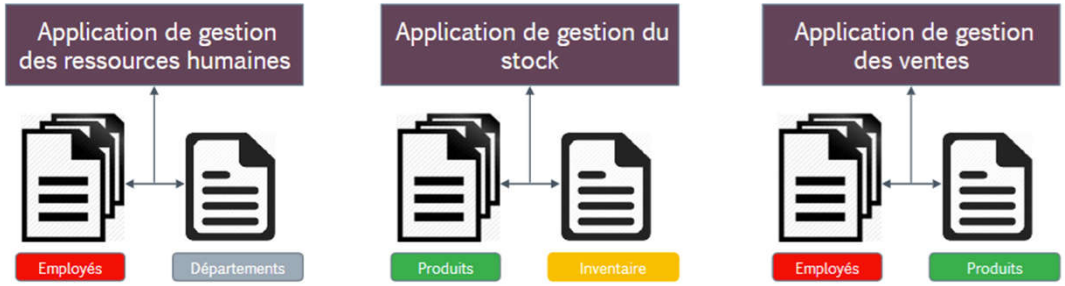
Plan

- Historique des bases de données
- Définition du NOSQL
- Caractéristiques des modèles NoSQL
- Relationnel vs NOSQL
- Théorème CAP
- Types de bases de données NOSQL
- Les bases de données NEWSQL

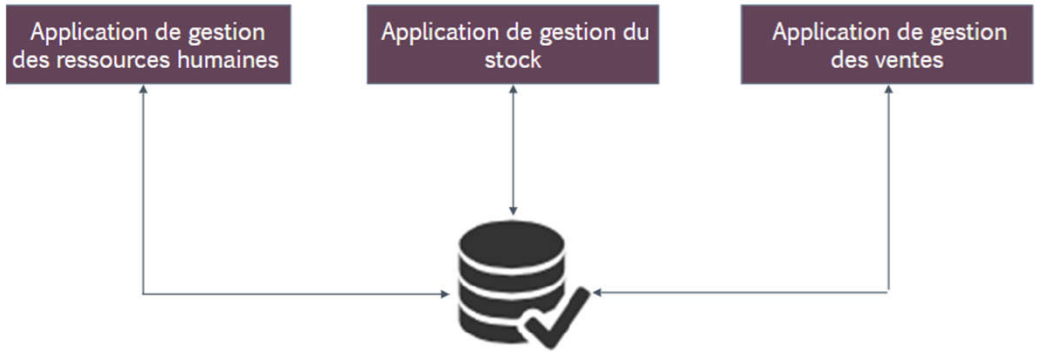
5 Historique des bases de données

Définition d'une base de données

- Une base de données est une collection structurée de données qui est organisée de manière à être facilement accessible, gérée et mise à jour.
- Elle est utilisée par les organisations comme méthode de stockage, de gestion et de récupération de l'information.

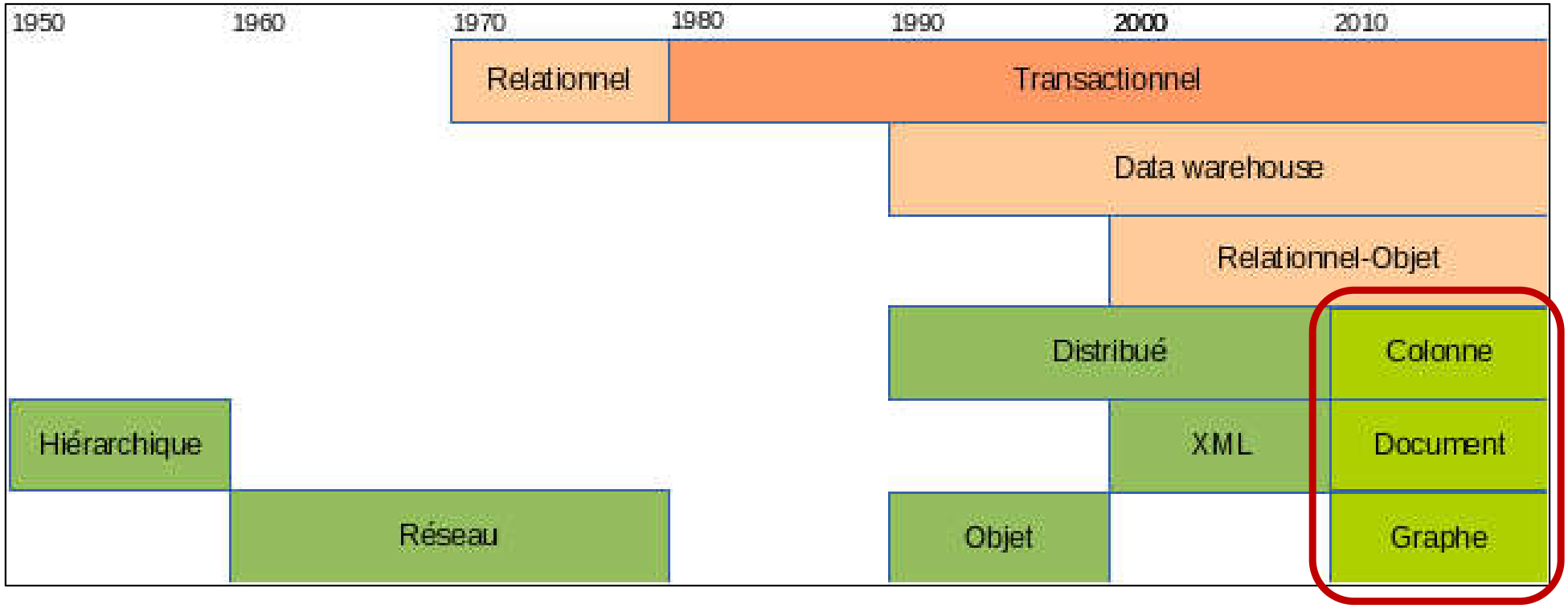


Approche sans BD



Approche avec BD

Vision Historique des bases de données

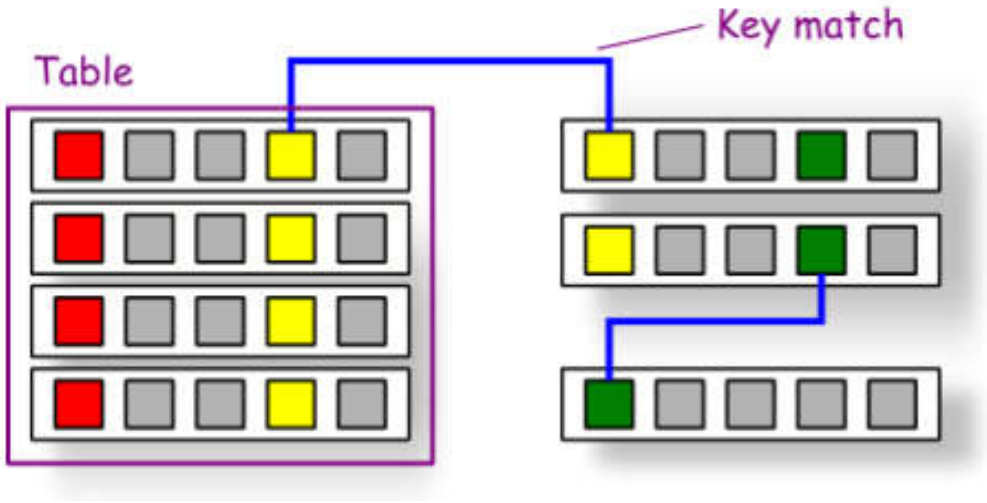


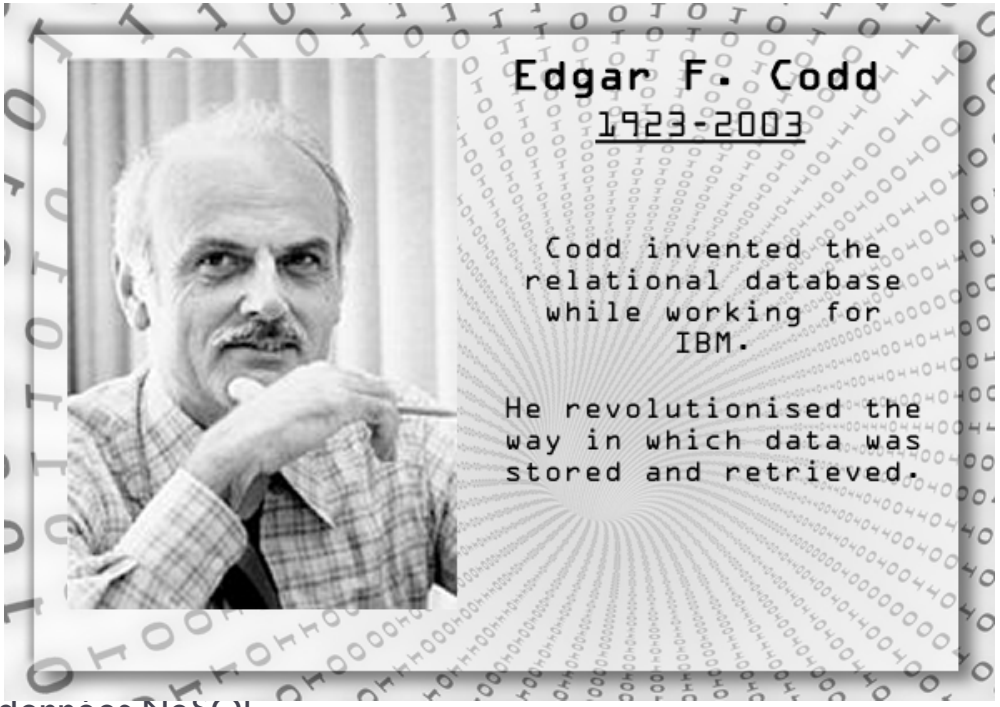
- Les BD NoSQL remettent en cause l'hégémonie des SGBDR telle qu'elle s'est constituée dans les années 1980.
- Les BD NoSQL sont essentiellement un retour à des modèles de données antérieurs (hiérarchique, réseau,...) qui existaient dans les années 60.

Vision Historique des bases de données:

-Apparition des bases relationnelles-

- À la naissance de l'informatique, plusieurs modèles de stockage de l'information sont explorés, comme les modèles: hiérarchique ou réseau.
- Finalement, le modèle relationnel qui l'emporte dans les années 1970 car c'est lui qui permet de mieux assurer le contrôle de l'intégrité des données, grâce à un modèle théorique puissant et simple.





Vision Historique des bases de données:

-Dominance du relationnel-

Forces du modèle relationnel :

Le schéma :

- Le schéma relationnel permet de définir des règles de cohérence explicites (intégrité référentielle, unicité, valeurs obligatoires).
- Ces règles sont intégrées dans le système et automatiquement contrôlées, réduisant les erreurs liées aux données invalides.

La normalisation :

- La normalisation organise les données en minimisant la redondance et en préservant leur intégrité.
- Elle repose sur un processus de décomposition en relations plus simples, tout en permettant de reconstituer l'information par des jointures.

La transaction :

- le système assure le maintien d'états cohérents au sein d'environnements concurrents et susceptibles de pannes.

Vision Historique des bases de données:

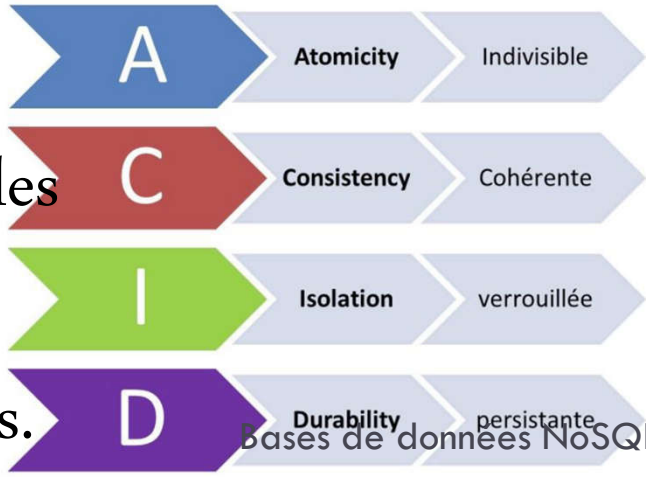
-Dominance du relationnel-

Définition d'une transaction:

Une transaction est un ensemble de modifications de la base qui forme un tout indivisible.

Caractéristiques d'une transaction: propriétés ACID (Gray, 1981)

- **Atomicité:** Tout ou rien : les actions de la transaction sont entièrement réalisées ou pas du tout, pas d'intermédiaire à moitié fait
- **Cohérence** (dépend de l'application) permet de passer d'un état cohérent à un autre état cohérent
- **Isolation:** Les transactions n'interfèrent pas entre elles
- **Durabilité** (ou permanence)une fois la transaction validée (rendue définitive), ses résultats sont préservés.



Vision Historique des bases de données:

-Dominance du relationnel-

SGBD relationnel (SGBDR) :

- **Base théorique :** modèle relationnel (indépendant de la structure de stockage)
- **Gestion de l'intégrité des données,** de la concurrence et de la reprise sur panne
- **Gestion et optimisation des requêtes** : indépendance entre la déclaration des requêtes et leur exécution, mécanismes d'indexation...
- **Outils matures** permettant de gérer de grands volumes de données (gigaoctet, voire téraoctet)
- **Outils de mapping objet-relationnel** (ORM) : JPA, Hibernate ...

Histoire des bases de données:

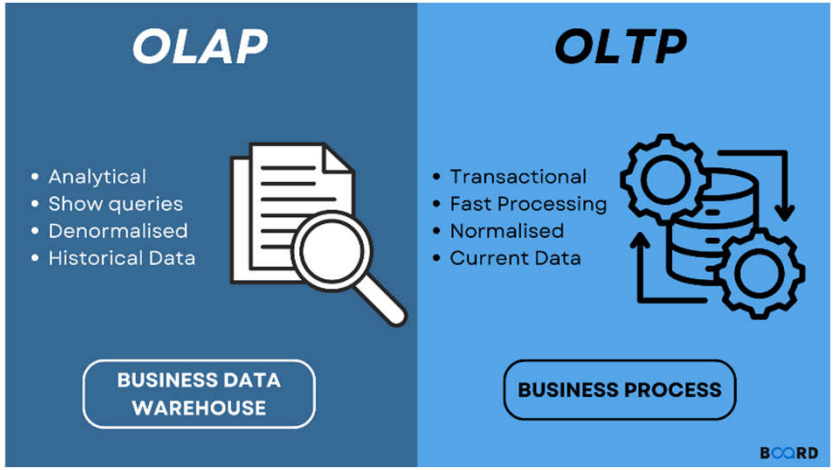
- Transactionnel vs Décisionnel-

- La représentation relationnelle se fonde sur la décomposition de l'information:
  - minimiser les entrées/sorties (accès disques, transfert réseau)
  - Consolidation par jointures
  - performance pour répondre à des questions et des mises à jour ciblées (qui concernent **peu de données** parmi un ensemble qui peut être très grand).
  - C'est donc une bonne solution dans un **contexte transactionnel** qui comprend de nombreux accès ciblés à la base.
  - On parle de traitement transactionnel en ligne : OLTP (*Online Transactional processing*).
  - Cas d'utilisation des BD transactionnelles : systèmes commerciaux, systèmes bancaires, application de réservation des voyages….

Histoire des bases de données:

- Transactionnel vs Décisionnel-

- Les BDs transactionnelles ne sont pas une bonne solution pour des accès globaux à la base comme les statistiques sur les tendances de ventes.
  - Beaucoup de jointures pour reconsolider l'ensemble de l'information.
  - C'est le problème posé notamment par le décisionnel.
  - Modèle OLAP pour le traitement (pour OnLine Analytical Processing)



Vision Historique des bases de données:

-Transactionnel vs Décisionnel-

Système d'information décisionnel

« *Le système d'information décisionnel est un ensemble de données organisées de façon spécifiques, facilement accessibles et appropriées à la prise de décision [...].*

La finalité d'un système décisionnel est le pilotage d'entreprise.

*Les systèmes de **gestion** sont dédiés **aux métiers** de l'entreprise [...].*

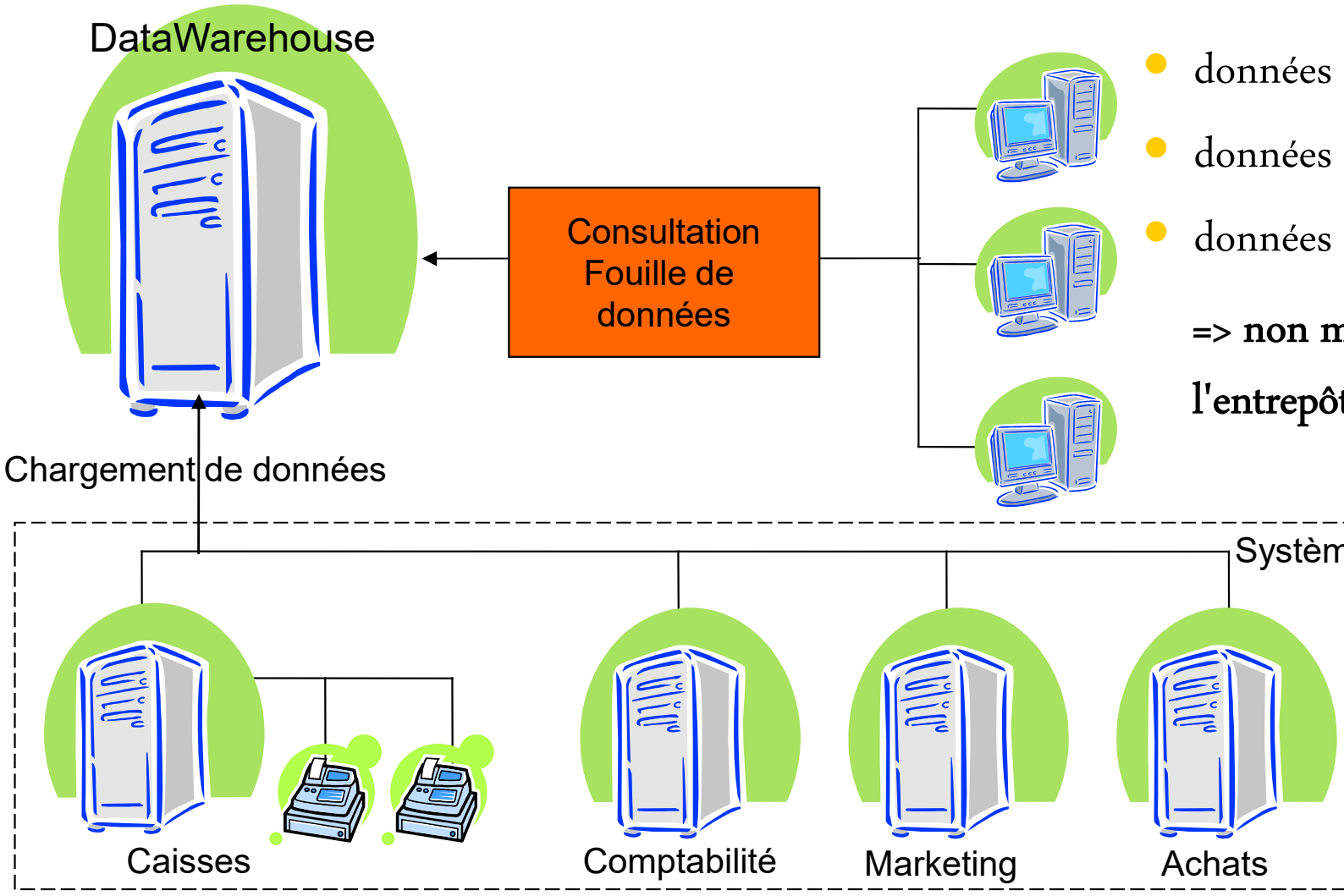
*Les systèmes **décisionnels** sont dédiés **au management** de l'entreprise [...].* »

(Goglin, 2001, pp21-22)

BD décisionnelle et «Datawarehouse »

- Un datawarehouse (DW) est une base de données construite par copie et réorganisation de multiples sources (principalement le système transactionnel), afin de servir de source de données à des applications décisionnelles :
  - il agrège de nombreuses données de l'entreprise (**intégration**);
  - il mémorise les données dans le temps (**historisation**);
  - il les organise pour faciliter les requêtes de prise de décision (**optimisation**).
  - Synonymes : entrepôt de données, base de données décisionnelle.
- Objectif: permettre des requêtes sur de grands ensembles des données, la plupart du temps sous forme d'agrégats (**GROUP BY**) afin d'en obtenir une vision synthétique (propre à la prise de décision).

BD décisionnelle et « datawarehouse »



Aide à la décision dans l'entreprise :

- données hétérogènes agrégées
- données thématiques
- données « historisées » :

=> non modifiables dans l'entrepôt de données

Vision Historique des bases de données:

-Vers le Big Data-

Evolution des technologies de BdD traditionnelles

Travail de CODD sur la représentation des données - 1969 → Modèle Relationnel 1970 → Avènement des SGBD relationnels - 1980 → Le stockage devient bon marché, les historiques de data abondent deviennent abondant → Les besoins en Data Analytics deviennent fréquents → BdD OLAP → BdD OLAP en mémoire et distribuées → **Approche Big data**

Approche Big data → Challenges in Big Data Analytics

Approche Big data → Challenges in Big Data Engineering

Deux types de requêtes / d'utilisation :

Requêtes OLTP : approche transactionnelle classique des BdD Relationnelles



Requêtes OLAP : besoin en analyse de données, peu favorable aux BdD Relationnelles



Définition du NOSQL

Limites des BDs relationnelles

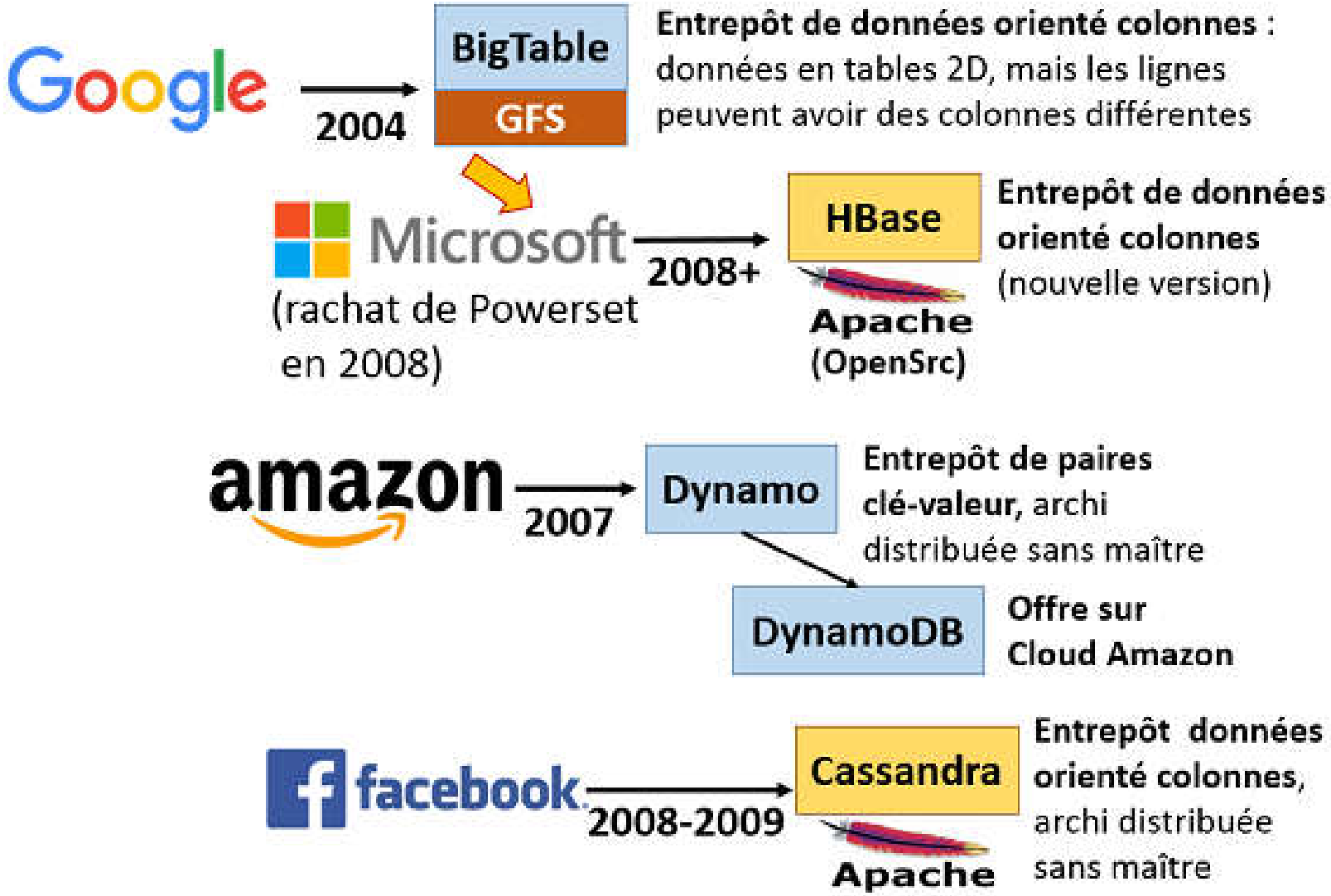
- Modèle relationnel (représentation tabulaire) parfois peu adapté au stockage et à l'interrogation de certains types de données : hiérarchiques, faiblement structurées, semi-structurées, complexes et hétérogènes
- Problème *d'Impedance mismatch* (Défaut d'impédance) : Non correspondance des modèles objet et relationnel.
- Systèmes généralement centralisés et non adaptés aux environnements distribués.
- Problèmes de gestion de très grands volumes de données (de l'ordre du pétaoctet) et de débits extrêmes (> milliers de requêtes par seconde)- Propriétés ACID : Surcoût en latence, accès disques, temps CPU (verrous, journalisation, etc.)
- Performances limitées par les accès disque

Emergence du NOSQL: Big Data

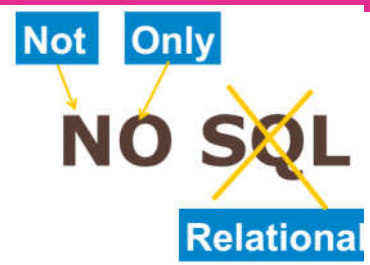
- **Origine :** recherche d'information sur le Web, moteurs type Google, Yahoo, données des réseaux sociaux,...
  - Besoin de stockage **d'énormes masses de données**.
  - Twitter par exemple reçoit plusieurs Tera-octets de données par jour.
  - Système distribué
  - table d'associations - Map - de couples (clef, valeur)
  - Différentes approches, rangées dans la famille "NOSQL".
- Ces technologies sont passées des acteurs du web au « grand public »

  →NOSQL n'est qu'une partie de cette vaste problématique du Big Data..

NOSQL: Partie du Big Data

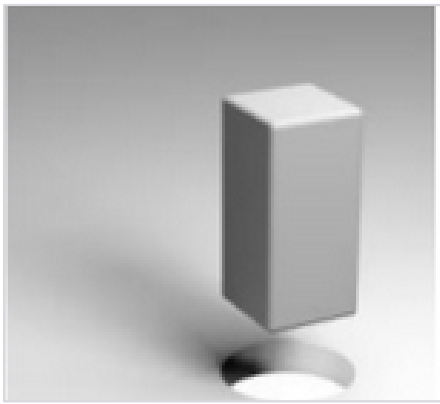


NOSQL: C'est quoi donc?



- Nom exact: Bases de données non relationnelles.
- Privilégier NOSQL plutôt que NoSQL.
- Terme (ré)apparu en 2009 vague et incorrect :certains moteurs NoSQL utilisent des variantes du SQL)
- Ce n'est pas du relationnel, et le contexte d'utilisation n'est pas le même.
- Ce n'est pas seulement l'opposition à SQL :
  - il s'agit de compléments aux SGBDR pour des besoins spécifiques et non de solutions de remplacement.
  - nouveaux besoins, nouveaux outils !
  - logiciels de stockage de données plutôt que SGBD

NOSQL: C'est quoi donc?



"The whole point of seeking alternatives [to RDBMS systems] is that you need to solve a problem that relational databases are a bad fit for."

Eric Evans
Rackspace

- Un couple de concepteurs de bases de données entre dans un restaurant NOSQL. Une heure après, ils ressortent sans avoir réussi à manger.
  - Pourquoi?



Caractéristiques des modèles NOSQL

Caractéristiques des modèles NOSQL

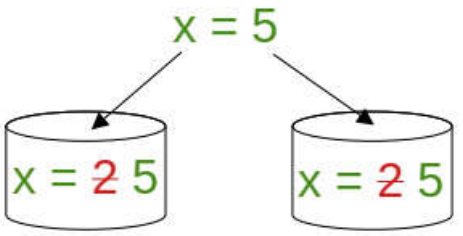
- Unité logique de stockage : AGREGAT et non plus relation (table)
  - **Agrégat**: collection d'objets référencés par une clé (pas d'autres d'accès que par la clé)
  - **Pas de liens entre les agrégats** (contrairement aux relations avec les contraintes d'intégrité référentielles) : dénormalisation
  - Indépendance de chaque agrégat d'où une répartition possible des données
  - A utiliser en cas de :
    - Grand volume de données
    - Architecture fortement répartie (distributed)
    - Requêtes massives
    - Schéma évolutif

Caractéristiques des modèles NOSQL

- **Imbrication:** Structure des valeurs stockées connue par le serveur (JSON, XML, structure interne de type colonne...)
- **Logique de dépôt uniquement** (stocker et retrouver):
  - c'est la couche applicative qui fait tout le travail (traitement, cohérence...).
- **Identification:** Les clés d'identification sont des clés artificielles, en général unique à l'échelle de la BD, voire du monde :
  - Object Identifiers (OID)
  - Universally Unique IDentifier (UUID)
  - Uniform Resource Identifier (URI)

Caractéristiques des modèles NOSQL

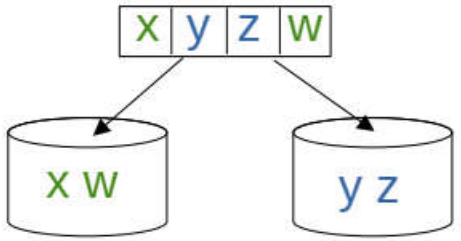
- **Schema-less:** Les bases NOSQL se fondent sur une approche dite *schema-less*, c'est à dire sans schéma logique défini à priori.
  - Souplesse et rapidité, mais moins de contrôle et donc de cohérence des données.
  - Le mouvement NOSQL tend à réintégrer des fonctions de schématisation à priori (comme pour XML): le schéma est optionnel, mais conseillé en contexte de contrôle de cohérence.
- **Réplication des données pour tolérance aux pannes**:
  - Quand un serveur tombe, on bascule sur l'autre
  - Nécessite de faire les mêmes modifications sur les 2 serveurs .
  - Cohérence constante du contenu des deux BDD



réplication des données

Caractéristiques des modèles NOSQL

- **Partitionnement et distribution (sharding):** Les données sont partitionnées horizontalement et distribuées sur les nœuds d'un cluster. On utilise par exemple une clé de hashage pour distribuer les données.



distribution/partitionnement

- Exemple de données partitionnées: Transfert d'argent d'un compte d'une banque vers une autre banque
  - Requiert un débit sur un serveur puis un crédit sur un autre serveur
  - Doit faire les deux actions ou aucune pour avoir un état cohérent
  - Débiter (banque A, #1244, 1000€)
  - Créditer (banque B, #8812, 1000€)

NOSQL: fin du relationnel?

- BDR ne doit pas mourir mais des solutions de stockage doivent être envisagées pour des applications particulières (applications web en particulier)
  - Besoins des BDRs: schémas, cohérence forte, transactions,
  - Avantages des BDRs : matures et opérationnelles, expertise disponible, puissantes et stables.
  - ➢ Pas près de disparaitre à court et moyen terme
- ...mais elles ne sont plus suffisantes
  - à cause de leur lourdeur inutile pour le stockage de certaines données
  - Solution: tendance des systèmes hybrides, polyglotte combinant plusieurs technologies : de manière concurrente, coopérative, répartie, redondante

Quand utiliser NOSQL?

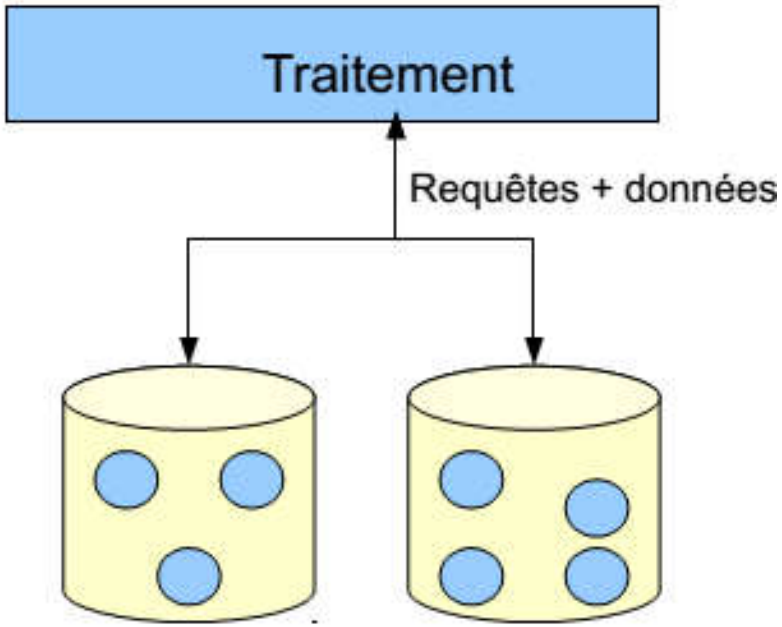
- Grand volume de données:
  - Besoin d'utiliser une architecture fortement distribuée
  - Google, Amazon, Yahoo, Facebook : 10-100K serveurs
- Requêtes massives:
  - Eviter les jointures car très gourmandes en temps de traitement
- Schéma évolutif:
  - Flexibilité et évolutivité du schéma à grande échelle
- Pas besoin de : transaction/ forte cohérence/intégrité/ requêtes complexes

Relationnel vs NOSQL

Relationnel vs NOSQL

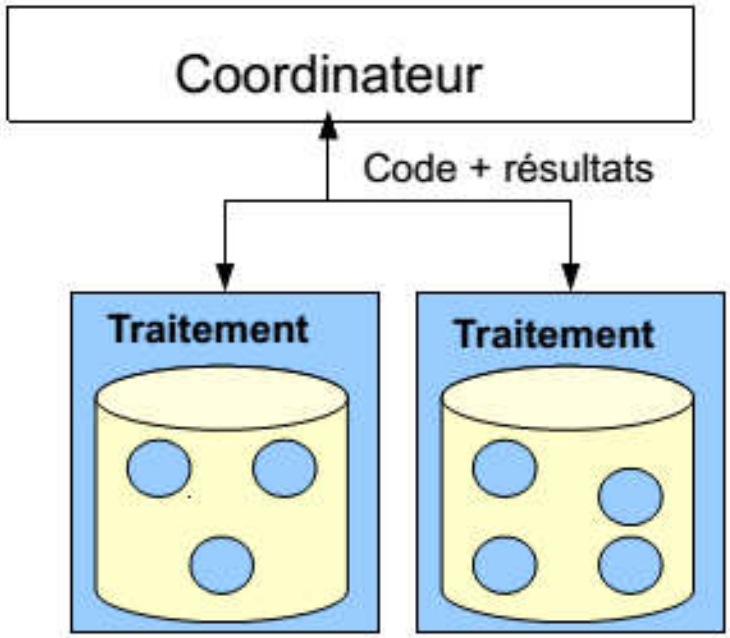
Relationnel

- Traitements centralisés
- Accès distribué



NOSQL

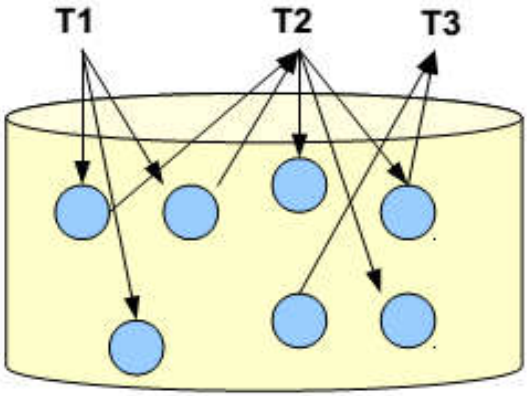
- Traitements distribués
- Accès local



Relationnel vs NOSQL

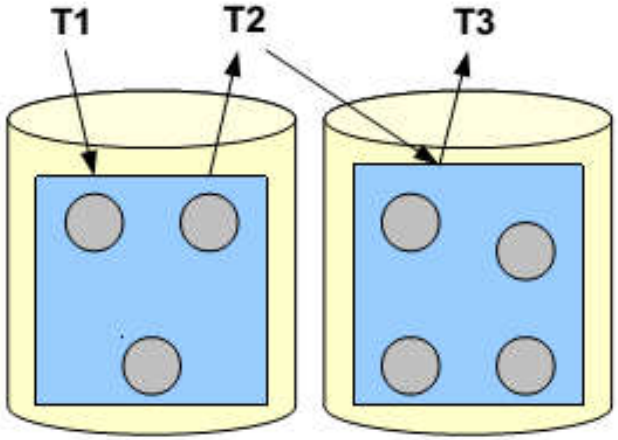
Relationnel

- Accès à grain fin:
  - beaucoup de lectures/écritures de petits objets.



NOSQL

- Accès « batch » :
  - peu de lectures/ écritures de grands objets



Relationnel vs NOSQL (ACID vs Base)

Propriétés ACID pour les transactions

- **A**tomicité : une transaction s'effectue entièrement ou pas du tout
- **C**ohérence : le contenu d'une base doit être cohérent au début et à la fin d'une transaction.
- **I**solation : les modifications d'une transaction ne sont visibles/modifiables que quand celle-ci a validé.
- **D**urabilité : une fois la transaction validée, l'état de la base est permanent (non affecté par les pannes ou autre.)

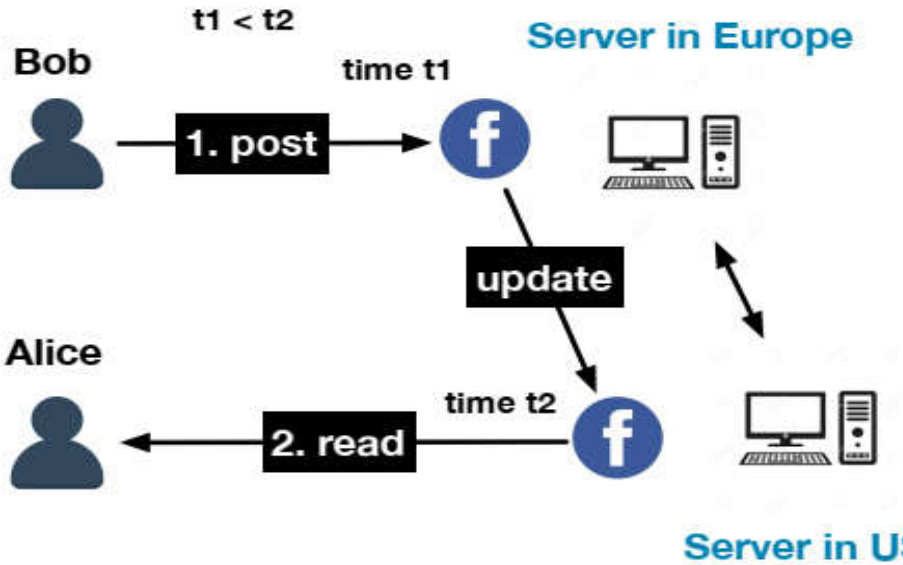
Systèmes distribués : modèle BASE

- **B**asically **A**vailable : quelle que soit la charge de la base de données, le système garantie un taux de disponibilité de la donnée
- **S**oft-state : l'état du système peut changer au cours du temps même sans nouveaux inputs (cela est dû au modèle de consistance).
- **E**ventually consistent (finalement consistent) : tous les réplicas atteignent le même état, et le système devient à un moment consistant, si on stoppe les inputs.



ACID ou BASE ?

- Choisir selon le cas d'utilisation.
- Le modèle ACID est parfois trop contraignant :
  - Système de réservation de billets d'avion: Modèle ACID nécessaire.
  - Calcul du nombre de messages envoyé par utilisateurs de chaque tranche d'âge, sur Facebook ? Modèle ACID inutile.
  - **Exemple:**

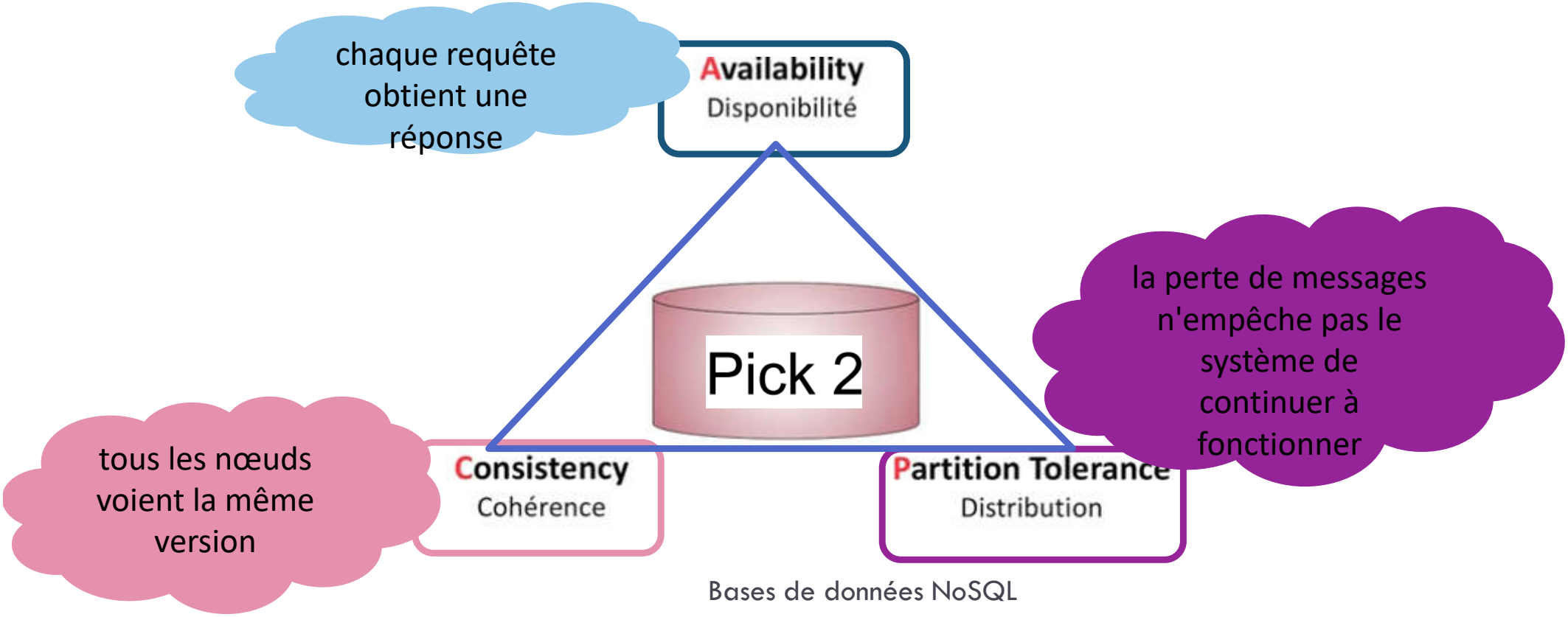


Alice does not see Bob's post between $t1$ and $t2$.
Is it really an issue?

Théorème CAP

Théorème CAP

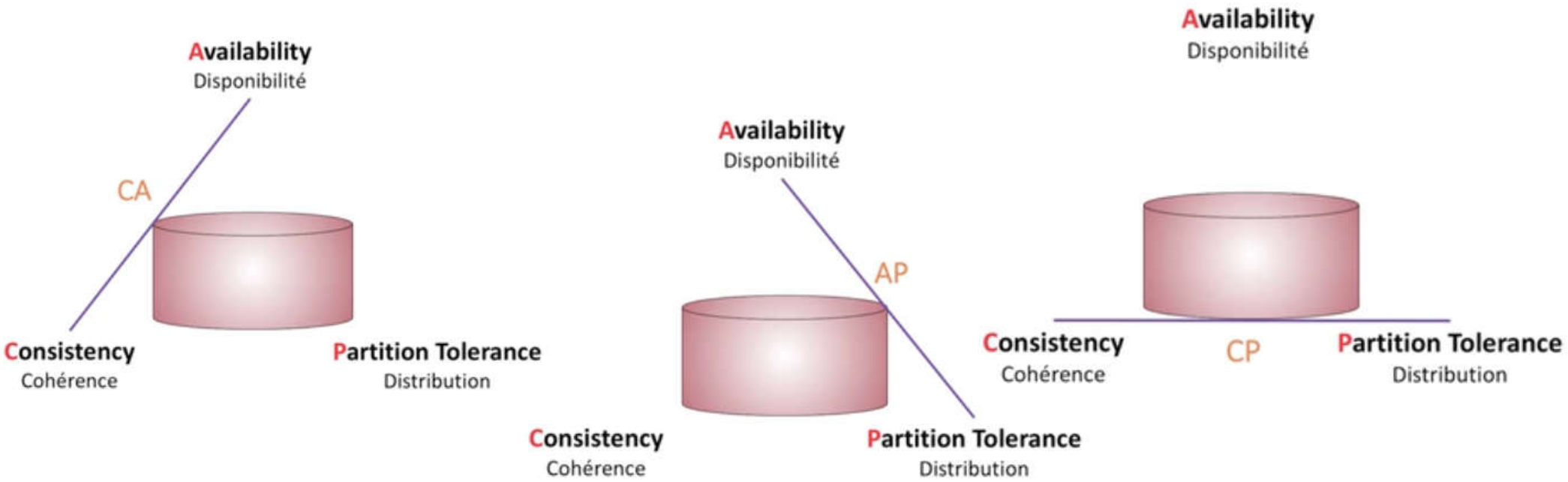
- En 2000, Eric A. Brewer a formalisé un théorème très intéressant reposant sur **3 propriétés fondamentales** pour caractériser les bases de données (relationnelles, NoSQL et autres) :



Théorème CAP

Au maximum **2 propriétés** respectées par n'importe quel système de données.

- plus précisément, si une garantie haute est assurée pour deux des propriétés, une garantie plus faible est assurée pour la dernière.

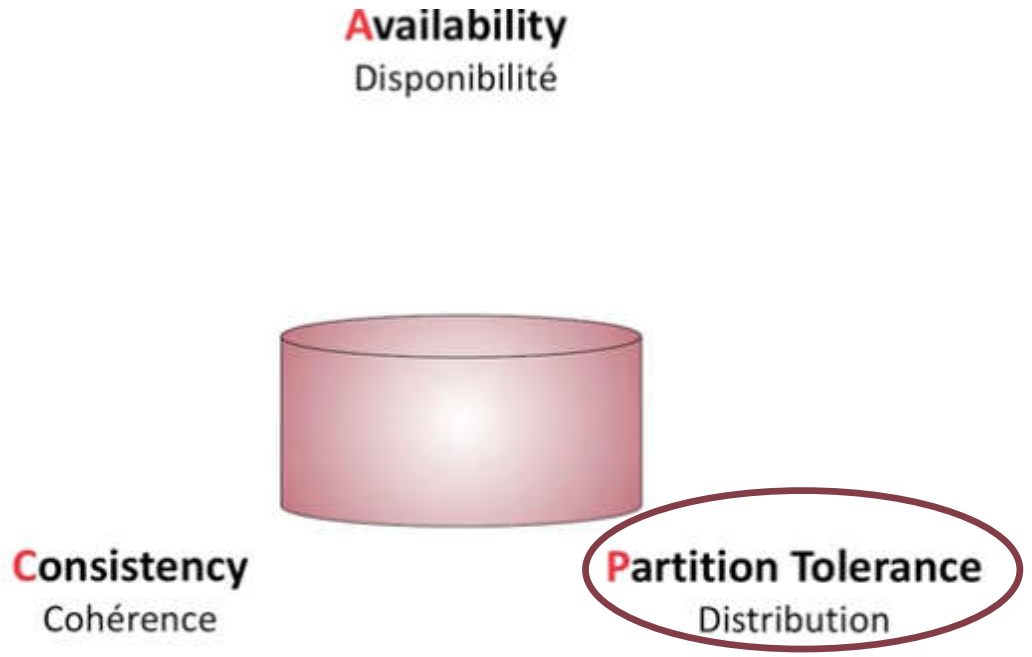


Théorème CAP

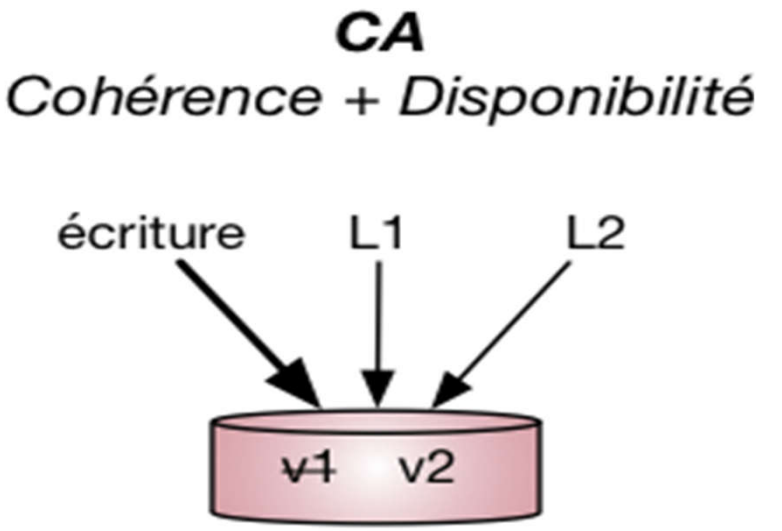
Au maximum **2 propriétés** respectées par n'importe quel système de données distribué:

- plus précisément, si une garantie haute est assurée pour deux des propriétés, une garantie plus faible est assurée pour la dernière.

- Pour passer à l'échelle, on utilise le **partitionnement.** Ce qui implique un **choix entre la cohérence ou la disponibilité.**

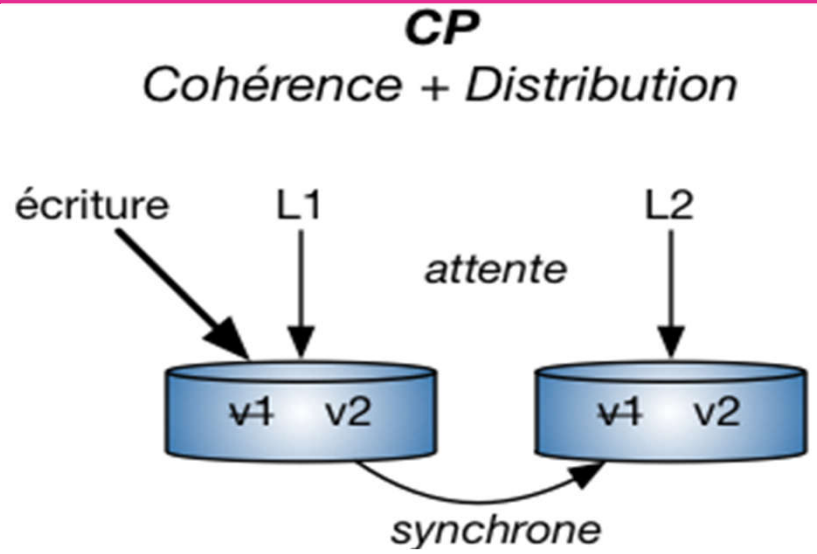


Théorème CAP



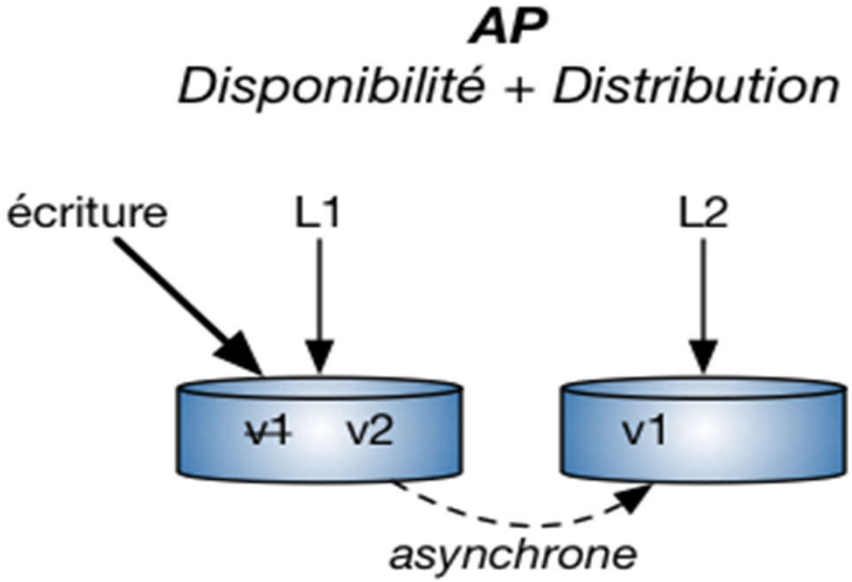
Lors d'opérations concurrentes sur une même donnée, les requêtes L1 et L2 retournent la nouvelle version (v2) et sans délai d'attente. Cette combinaison n'est possible que dans le cadre de bases de données transactionnelles telles que les SGBDR.

Théorème CAP



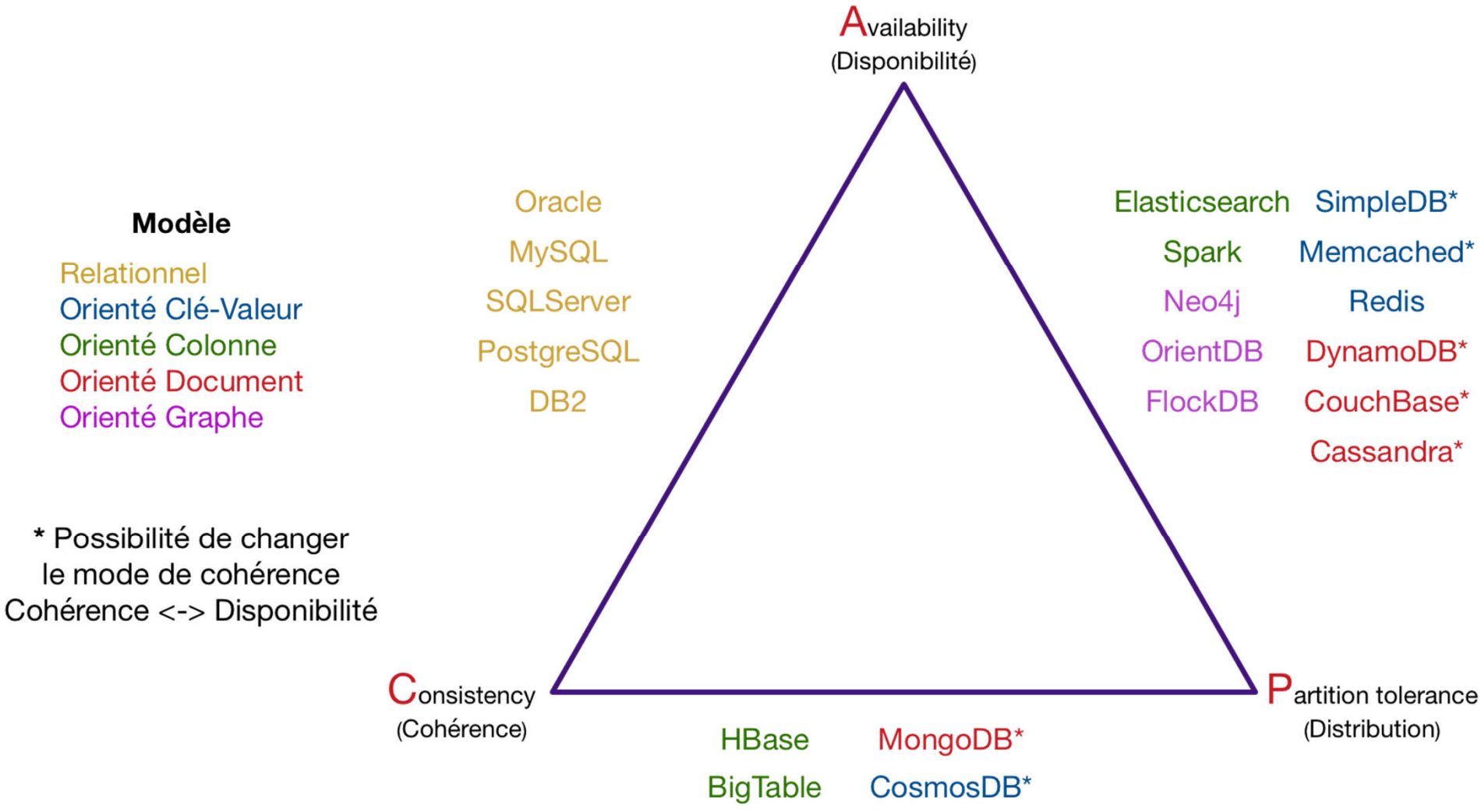
En même temps, il est nécessaire de vérifier la cohérence des données en garantissant la valeur retournée malgré des mises à jour concurrentielles. La gestion de cette cohérence nécessite un protocole de synchronisation des réplicas, introduisant des délais de latence dans les temps de réponse (L1 et L2 attendent la synchronisation pour voir v2). C'est le cas de la base NoSQL MongoDB.

Théorème CAP



AP à contrario s'intéresse à fournir un temps de réponse rapide tout en distribuant les données et les réplicas. De fait, les mises à jour sont asynchrones sur le réseau, et la donnée est "Eventually Consistent" (L1 voit la version v2, tandis que L2 voit la version v1). C'est le cas de Cassandra.

Classification des bases de données selon CAP



Top NOSQL Databases (Popularié)

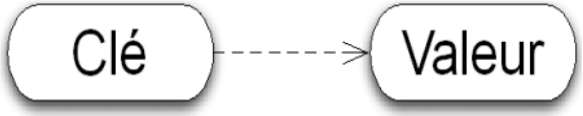
423 systems in ranking, December 2024

Rank Dec 2024	Rank Nov 2024	Rank Dec 2023	DBMS	Database Model	Score Dec 2024	Score Nov 2024	Score Dec 2023
1.	1.	1.	Oracle	Relational, Multi-model	1263.79	-53.22	+6.38
2.	2.	2.	MySQL	Relational, Multi-model	1003.76	-14.04	-122.88
3.	3.	3.	Microsoft SQL Server	Relational, Multi-model	805.69	+5.88	-98.14
4.	4.	4.	PostgreSQL	Relational, Multi-model	666.37	+12.04	+15.47
5.	5.	5.	MongoDB	Document, Multi-model	400.39	-0.54	-18.76
6.	6.	6.	Redis	Key-value, Multi-model	150.27	+1.63	-8.08
7.	7.	↑ 10.	Snowflake	Relational	147.36	+4.87	+27.48
8.	8.	↓ 7.	Elasticsearch	Multi-model	132.32	+0.68	-5.43
9.	9.	↓ 8.	IBM Db2	Relational, Multi-model	122.78	+1.04	-11.81
10.	10.	↑ 11.	SQLite	Relational	101.72	+2.24	-16.23
11.	11.	↑ 12.	Apache Cassandra	Wide column, Multi-model	97.94	+0.22	-14.26
12.	12.	↓ 9.	Microsoft Access	Relational	90.82	-0.49	-30.93
13.	↑ 14.	↑ 17.	Databricks	Multi-model	87.68	+1.22	+7.37
14.	↓ 13.	14.	Splunk	Search engine	85.36	-3.11	-10.94
15.	15.	↓ 13.	MariaDB	Relational, Multi-model	83.77	+1.07	-16.66
16.	16.	↓ 15.	Microsoft Azure SQL Database	Relational, Multi-model	76.37	-0.16	-6.67
17.	17.	↓ 16.	Amazon DynamoDB	Multi-model	72.73	+0.33	-9.68

Types des bases de données NOSQL

Types des bases de données NOSQL

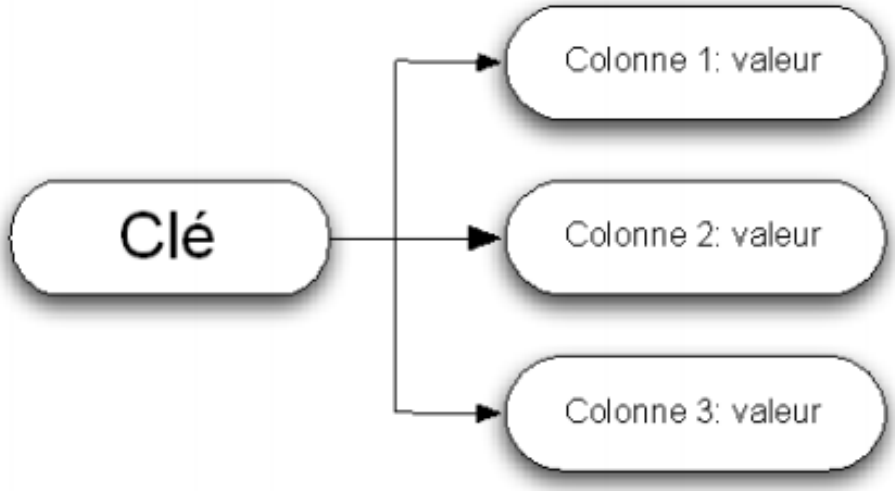
- 4 types des bases de données NOSQL:
  - Clef/valeur



redis

riak

  - Orientées colonnes

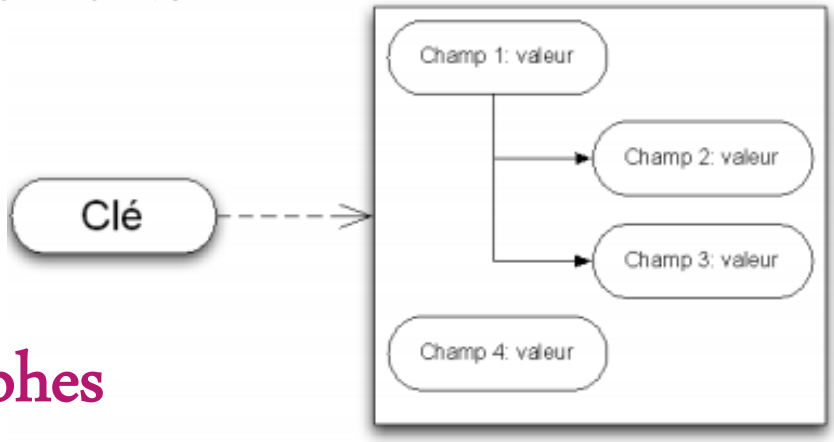


Cassandra

HBASE

Types des bases de données NOSQL

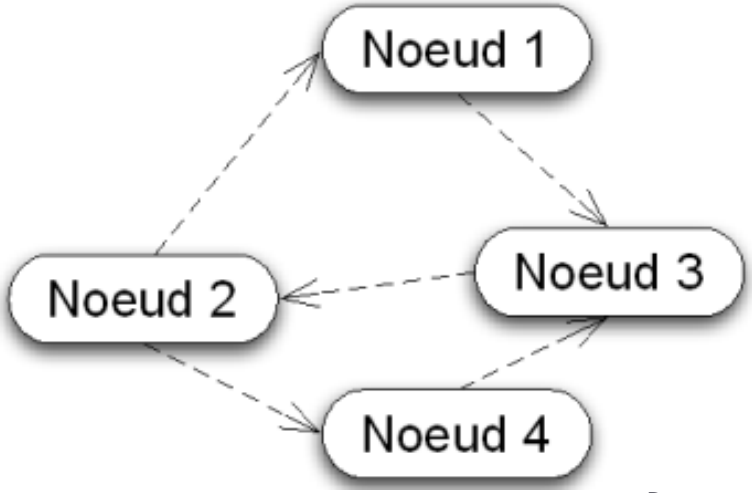
- 4 types des bases de données NOSQL:
  - Orientées documents



mongoDB

CouchDB relax

  - Orientées graphes

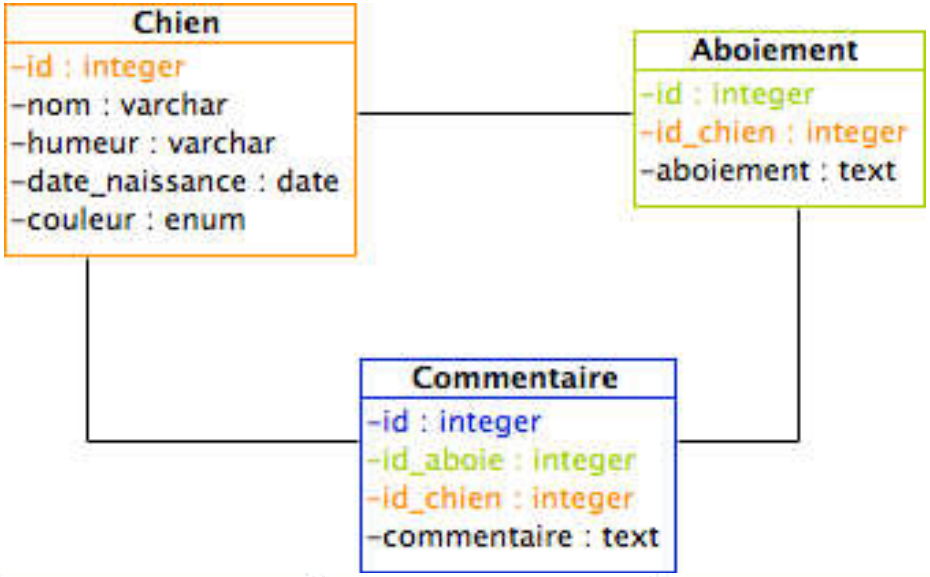


Neo4j the graph database

HyperGraphDB

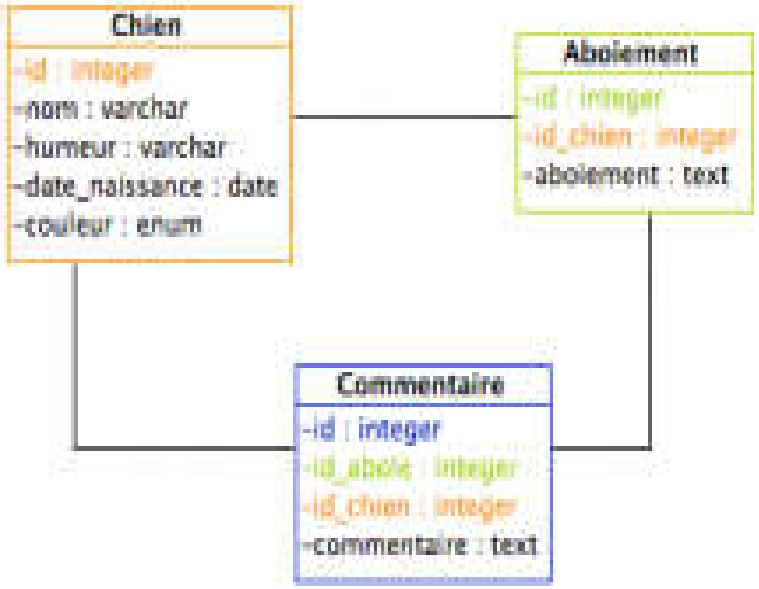
Types des bases de données NOSQL

- Exemple en relationnel:
  - NULL?

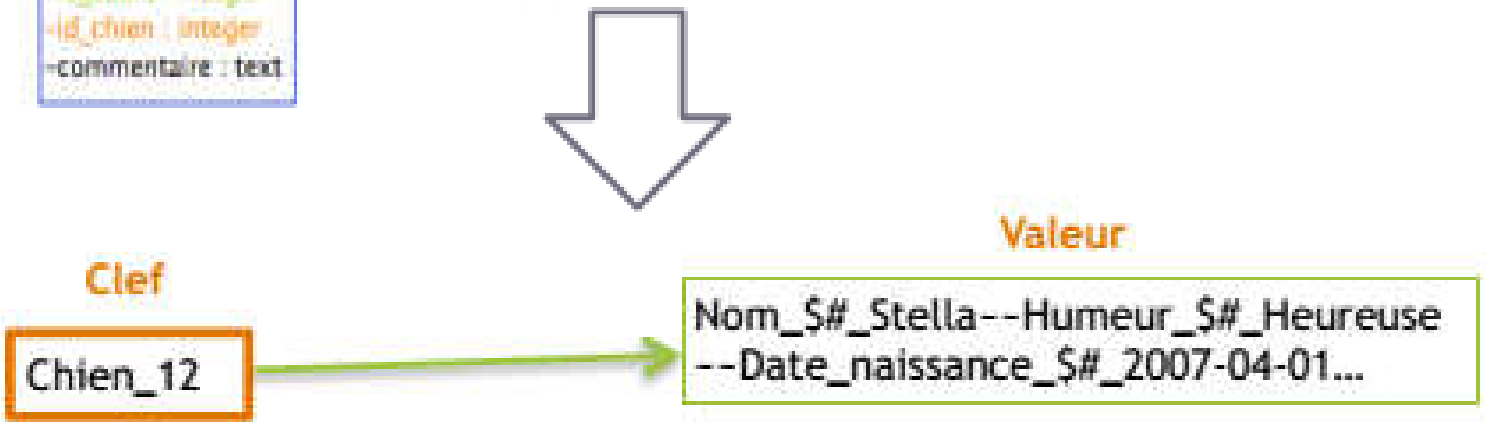


Id	Nom	Humeur	Date_naissance	Couleur
12	Stella	Heureuse	2007-04-01	NULL
13	Wimma	Faim	NULL	Noire
9	Ninja	NULL	NULL	NULL

1. Les basas de données Clef-valeur (Key/Value Store)



Id	Nom	Humeur	Date_naissance	Couleur
12	Stella	Heureuse	2007-04-01	NULL
13	Wimma	Faim	NULL	Noire
9	Ninja	NULL	NULL	NULL



1. Les bases de données Clef-valeur (Key/Value Store)

- L'un des types les plus simples, sorte de Hashmap distribuée
- Conçu pour sauvegarder les données sans définir de schéma.
- Toutes les données sont sous forme de clef/valeur
  - La valeur peut être une chaîne de caractères, un objet sérialisé, blob…
  - La donnée est opaque au système: il n'est pas possible d'y accéder sans passer par la clef
- Absence de typage a un impact sur le requêtage: toute l'intelligence portée avant par les requêtes devra être portée par l'applicatif qui interroge la BD.
- Communications se résumant surtout aux opérations PUT, GET et DELETE.
- Objectif : fournir un accès rapide aux informations, conserver la session d'un site web…
- Exemple : DynamoDB (Amazon), Azure Table Storage (ATS), Redis, BerkeleyDB, Voldemort (LinkedIn).

1. Les basas de données Clef-valeur (Key/Value Store)

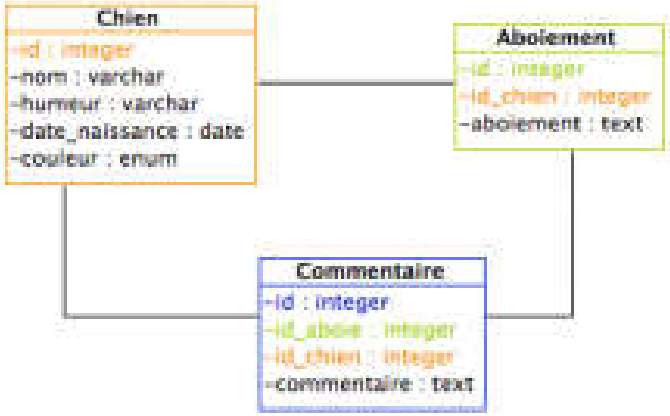
Avantages:

- Très rapide
- Passage à l'échelle
- Modèle simple
- Distribution horizontale très facile

Inconvénients:

- Interrogation seulement sur clé
- Modèle de données trop pauvre pour les données complexes
- Déporte une partie de la complexité au niveau de l'application

2. Les bases de données orientées Documents (Document Database)



Id	Nom	Humeur	Date_naissance	Couleur
12	Stella	Heureuse	2007-04-01	NULL
13	Wimma	Faim	NULL	Noire
9	Ninja	NULL	NULL	NULL

Clef

Chien_12

Document (V1)

```
{
        type :        « Chien »,
        nom : « Stella »,
        humeur :      « Heureuse »,
        date_naissance : 2007-04-01
}
```

Document (V2)

```
{
        type :    « Chien »,
        nom :     « Stella »,
        humeur :          « Heureuse »,
        date_naissance : 2007-04-01
        aboiement : [
                { texte : « j'ai mangé de la
pâtée »
                  commentaires : [
                        { id_chien : « chien_4 »,
                          texte : « on s'en fout! »
                        }
                  ]
        ]
}
```

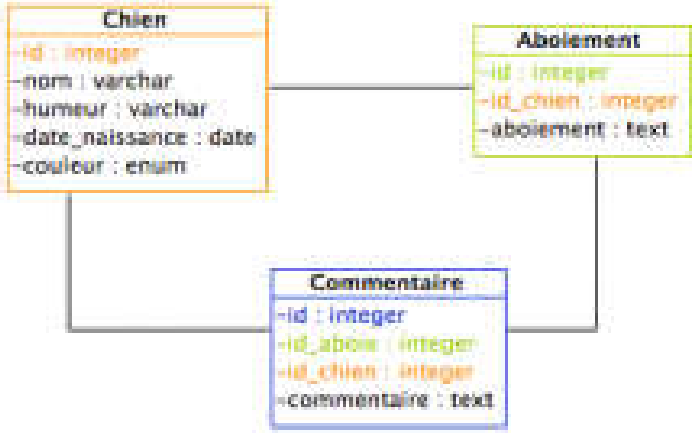
2. Les bases de données orientées Documents (Document Database)

- Les BD orientées document étendent le paradigme clef/valeur, avec des « documents » plus complexes à la place des données simples, et une clef unique pour chacun d'eux.
- Documents de type JSON ou XML.
- Chaque document est un objet, contient un ou plusieurs champs, et chaque champs contient une valeur typée (string, date, binary ou array)
- Permettent de stocker, extraire et gérer les informations orientées documents (données semi structurées)
- **Avantage :** pouvoir récupérer, via une seule clef, un ensemble d'informations structurées de manière hiérarchique
  - Dans les bases relationnelles, cela impliquerait plusieurs jointures
- **Exemples :** MongoDB (SourceForge), CouchDB (Apache), RavenDB (destiné aux plateformes .Net/ Windows).

2. Les bases de données orientées Documents (Document Database)

- **Avantages:**
  - modèle de données simple mais puissant
  - bon passage à l'échelle
  - Forte expressivité pour les requêtes (sur des structures imbriquées)
- **Inconvénients:**
  - inadapté pour les données interconnectées
  - peut-être lent pour les requêtes avec MapReduce

3. Les bases de données orientées Colonnes (Column Store)



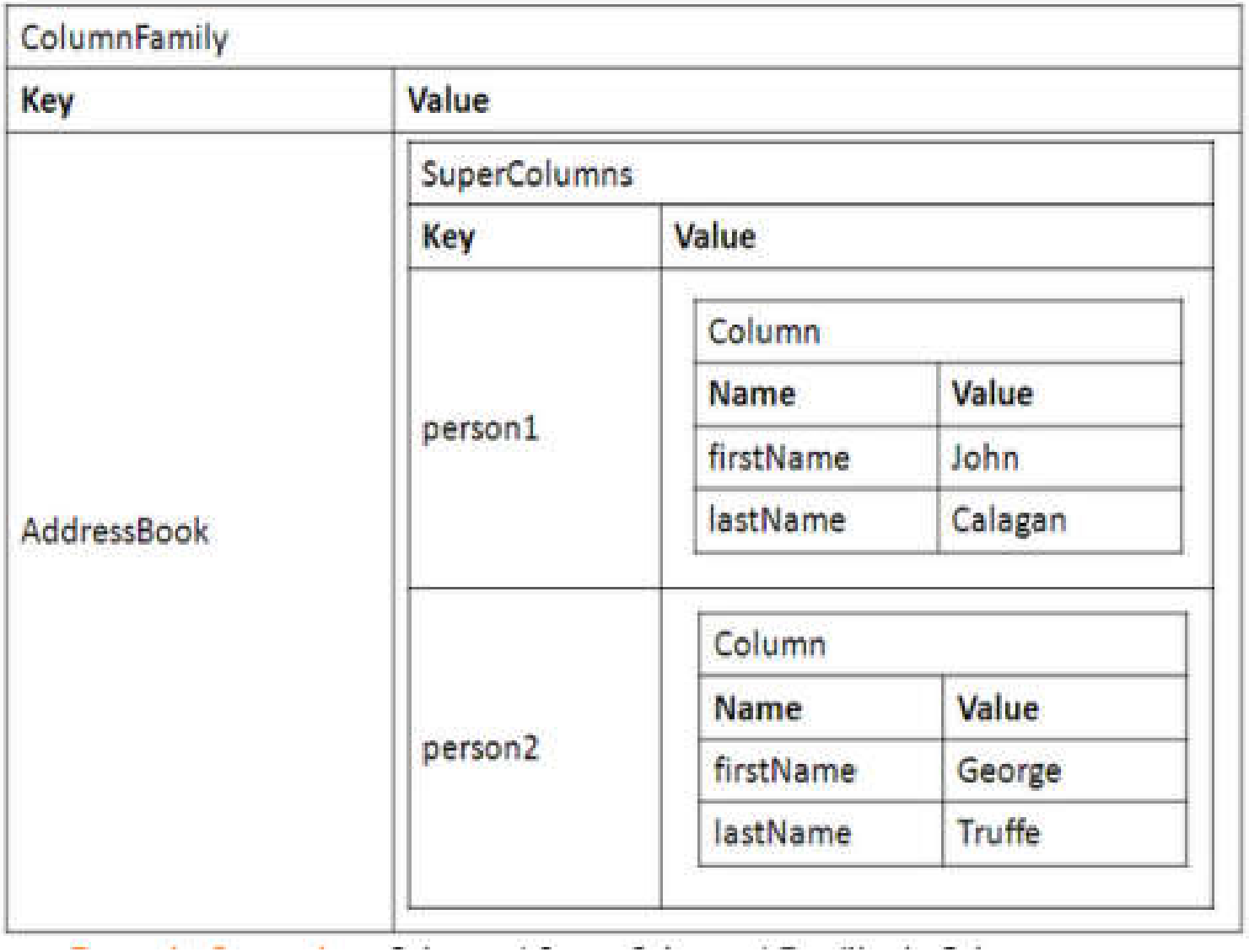
Id	Nom	Humeur	Date_naissance	Couleur
12	Stella	Heureuse	2007-04-01	NULL
13	Wimma	Faim	NULL	Noire
9	Ninja	NULL	NULL	NULL

Requête: clef/famille:titre[/time]
Exp: "chien_12"/"Chien":"Nom" → Stella

Colonnes

Clef	Famille	Titre	Temps	Valeur
Chien_12	Chien	Date_naissance	15	2007-04-01
	Chien	Humeur	11	En Colère
	Chien	Humeur	45	Heureuse
	Chien	Nom	25	Stella
	Chien	Couleur	34	Noire
	Aboiement	Text	11	J'ai mangé de la pâtée

3.Les bases de données orientées Colonnes (Column Store)



3. Les bases de données orientées Colonnes (Column Store)

- Évolution de la BD clef/valeur
- Ressemble aux SGBDR, mais avec un nombre de colonnes dynamique, différent d'un enregistrement à un autre (pas de colonnes portant les valeurs NULL)
- Offrent de très hautes performances et une architecture hautement évolutive
- Exemples: Hbase (Hadoop), Cassandra (Facebook, Twitter), BigTable (Google)

	A	B	C	D	E
1	Foo	Bar	Hello		
2		Tom			
3			Java	Scala	Cobol



3.Les bases de données orientées Colonnes (Column Store)

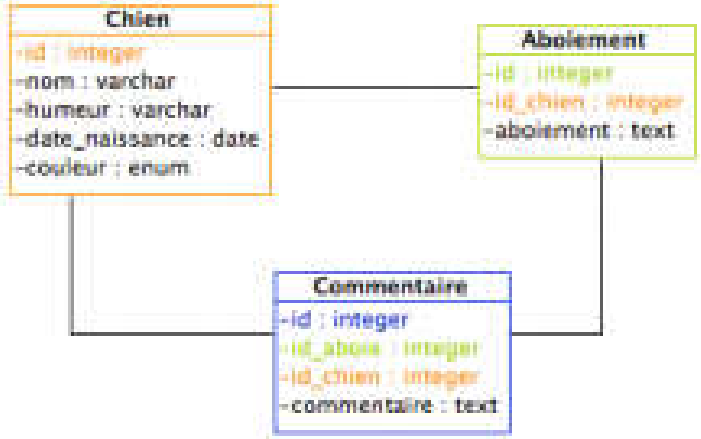
Avantages:

- modèle de données riche
- scalabilité horizontale et utilisation de MapReduce
- résultats des requêtes en temps réel

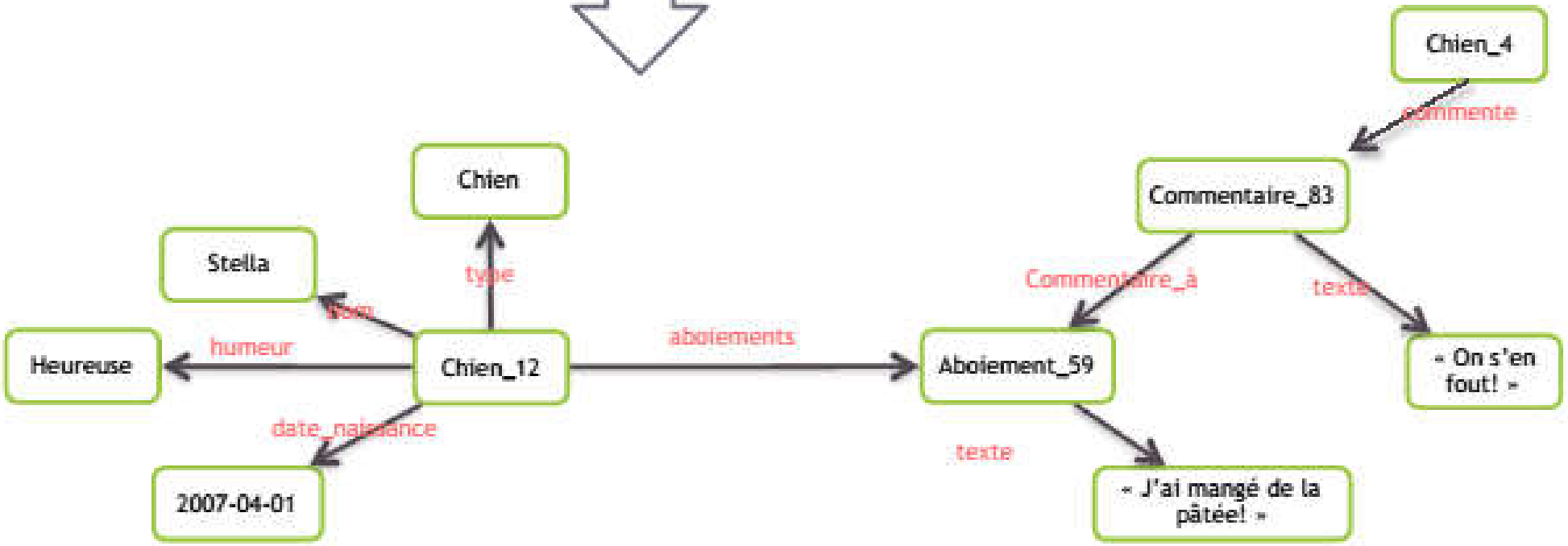
Inconvénients:

- difficile à utiliser pour les données interconnectés (distance, trajectoire)
- maintenance lors de l'ajout/suppression de colonnes
- ne pas utiliser pour les requêtes non temps réel et inconnues

4.Les bases de données orientée graphes (Graph Database)



Id	Nom	Humeur	Date_naissance	Couleur
12	Stella	Heureuse	2007-04-01	NULL
13	Wimma	Faim	NULL	Noire
9	Ninja	NULL	NULL	NULL



4.Les bases de données orientées graphes (Graph Database)

- Basées sur les théories des graphes.
- S'appuie sur les notions de nœuds, de relations et des propriétés qui leur sont rattachées.
- Conçues pour les données dont les relations sont représentées comme graphes, et ayant des éléments interconnectés, avec un nombre indéterminé de relations entre elles.
- Adapté aux traitements des données des réseaux sociaux.
- **Exemples :** Neo4J et InfiniteGraph, OrientDB.

4.Les bases de données orientée graphes (Graph Database)

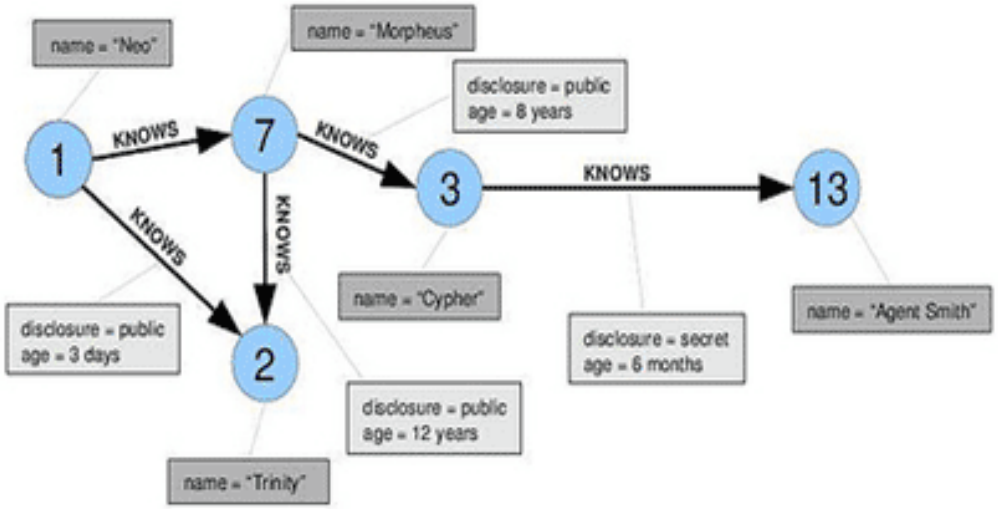
Avantages:

- modèle de données puissant
- rapide pour les données liées
- modèles d'interrogation performants : Cypher, SPARQL,GraphQL
- Algorithmes de la théorie des graphes, par exemple le plus court chemin

Inconvénients:

- fragmentation, possible seulement pour certains domaines
- concept relativement récent
- Doit traverser le graphe entier avant d'obtenir une réponse définitive

4.Les bases de données orientée graphes (Graph Database)



Adapté à la gestion de données relationnelles

Architecture modelable

Architecture limitée à certains cas

Exemple d'implémentation

TITAN

Neo4j